VIT BHOPAL
www.vitbhopal.ac.in

SCHOOL OF COMPUTING SCIENCE ENGINEERING AND ARTIFICIAL INTELLIGENCE

(SCOPE)

PROJECT PRACTICAL FILE

DISCRETE MATHEMATICS AND GRAPH THEORY

Course Code: MAT2002

SMART ROUTE FINDER

Shortest Path Analysis using Dijkstra's Algorithm

Class ID: BL2026270100016	**Slot:** B14+B23+D21
Submitted To: Course Faculty	**Academic Year:** 2026–2027

SUBMITTED BY:

S. No.	Student Name	Registration No.
1	Yash Janiyani	24BCE11394
2	Himanshu Vishwakarma	25MIM10119
3	Prashant Raj	24BCE11371
4	Abhishek	24BCE11402
5	Mradul Agrawal	24BCE11403
6	Devraj Srivastava	24BCE10050

TABLE OF CONTENTS / INDEX

S. No.	Section Title	Page No.
1	Project Overview	3
2	Aim of the Experiment	3
3	Problem Statement & Real-World Utility	3
4	Project Objectives	3
5	Mathematical & Theoretical Background	5
6	Dijkstra's Algorithm & Relaxation Mechanics	5
7	System & Hardware Requirements	6
8	Methodology & Application Workflow	6
9	Algorithm Pseudocode	6
10	Software Implementation & Code Snippets	7
11	Project Output & Screenshots	8
12	Sample Numerical Working	9
13	Result	9
14	Conclusion	9
15	Limitations & Future Scope	9
16	Top 20 Viva Preparation Questions & Answers	11
17	Team Member Details & Signatures	13

1. PROJECT OVERVIEW

Smart Route Finder is an interactive, web-based educational software application designed specifically for the university course **Discrete Mathematics and Graph Theory (MAT2002)**. The primary purpose of this project is to bridge theoretical discrete graph structures with practical software engineering by modeling a **weighted directed graph** and demonstrating how **Dijkstra's Shortest Path Algorithm** computes the optimal (minimum cost) route between a selected source vertex and destination vertex.

In modern computer science, graph theory provides the foundational data abstraction for network topology, routing protocols, and transportation systems. This project implements an intuitive, zero-dependency frontend application featuring interactive SVG graph visualization, dynamic adjacency matrix generation, and step-by-step distance relaxation logging to facilitate academic evaluation and viva defense.

2. AIM OF THE EXPERIMENT

"To design, develop, and mathematically validate an interactive web application that represents a weighted directed graph, dynamically generates its weighted adjacency matrix, and computes the shortest path between a user-defined source and destination vertex using a manual implementation of Dijkstra's Shortest Path Algorithm."

3. PROBLEM STATEMENT & REAL-WORLD UTILITY

Problem Statement: Given a weighted directed graph $G = (V, E)$ consisting of a finite set of vertices V and directed edges E with non-negative edge weights $w(u, v) > 0$, determine the minimum cumulative distance from a designated source vertex $s \in V$ to a destination vertex $d \in V$, along with the exact sequence of vertices traversed.

Real-World Applications:

• **Network Packet Routing:** Open Shortest Path First (OSPF) and IS-IS internet routing protocols utilize Dijkstra's algorithm to route IP packets through routers with minimum latency.

• **Transportation & Logistics:** Determining fuel-efficient cargo delivery paths across interconnected road networks with directed traffic rules.

• **Telecommunication Networks:** Channel allocation and minimal-cost fiber optic line transmission.

4. PROJECT OBJECTIVES

1. **Graph Representation:** Represent directed vertices and weighted edges in React frontend state without external database overhead.

2. **Manual Algorithm Implementation:** Code Dijkstra's algorithm manually from scratch to demonstrate full mastery of data structures and loops.

3. **Matrix Generation:** Dynamically compute and display the $V \times V$ weighted adjacency matrix matching the MAT2002 syllabus.

4. **Interactive Visualization:** Render draggable SVG circular nodes, directed arrows with arrowheads, and edge weight tags.

5. **Algorithm Transparency:** Provide a step-by-step trace log recording each vertex selection and distance relaxation.
6. **Data Validation:** Prevent negative weights, zero weights, duplicate vertices, duplicate directed edges, and self-loops.

5. MATHEMATICAL & THEORETICAL BACKGROUND

This project is anchored in fundamental discrete mathematics concepts from the MAT2002 syllabus:

5.1 Graph $G = (V, E)$: A mathematical structure consisting of a set of vertices $V = \{v_1, v_2, \dots, v_n\}$ and a set of edges $E = \{e_1, e_2, \dots, e_m\}$.

5.2 Vertex (Node): An individual discrete element representing an intersection, campus facility, or network router.

5.3 Edge (Link): A connection between two vertices representing a physical link or path.

5.4 Directed Graph (Digraph): A graph where each edge has an assigned direction (u, v). Traversal from $u \to v$ does not imply traversal from $v \to u$.

5.5 Weighted Graph: A graph where a non-negative function $w: E \to \mathbb{R}^+$ assigns a numerical cost/distance $w(u, v)$ to each edge.

5.6 Path: A sequence of vertices $\langle v_0, v_1, \dots, v_k \rangle$ such that $(v_{i-1}, v_i) \in E$ for $1 \le i \le k$. The total length of the path is $\sum_{i=1}^k w(v_{i-1}, v_i)$.

5.7 Shortest Path: A path from source s to destination d whose total weight is minimal among all possible paths connecting s and d.

5.8 Weighted Adjacency Matrix (A_{ij}): A $V \times V$ square matrix defined as:

```
$$A_{ij} = \begin{cases} 0 & \text{if } i = j \\ w(i, j) & \text{if } (i, j) \in E \\ \infty & \text{if } (i, j) \notin E \end{cases}$$
```

6. DIJKSTRA'S ALGORITHM & RELAXATION MECHANICS

Dijkstra's Algorithm is a greedy single-source shortest path algorithm suitable for graphs with non-negative edge weights. It maintains three core state structures:

1. **Distance Array (dist[]):** Holds tentative shortest distances from source s. Initialized to 0 for s and ∞ for all other vertices.

2. **Previous Pointer Array (prev[]):** Stores the preceding node along the current shortest path. Initialized to null.

3. **Unvisited Set (Q):** Set of unfinalized vertices, initially containing all vertices V.

Distance Relaxation Concept:

Relaxation is the process of testing whether passing through vertex u improves the current shortest distance to neighbor v. The mathematical triangular inequality update is:

```
If dist[u] + w(u, v) < dist[v] ■ dist[v] = dist[u] + w(u, v), prev[v] = u
```

Why Non-Negative Weights are Mandatory:

Dijkstra relies on the *greedy choice property*: once a vertex u with minimum tentative distance is selected from Q, its distance dist[u] is finalized and never re-evaluated. If negative edge weights existed, a longer path could later decrease in total weight, rendering greedy choices invalid. For negative weights, algorithms like Bellman-Ford must be used.

Time and Space Complexity:

• **Time Complexity:** $\mathcal{O}(V^2)$ using linear unvisited array scan (as implemented in dijkstra.js).

• **Space Complexity:** $\mathcal{O}(V + E)$ for storing adjacency lists, distance maps, and step trace buffers.

7. SYSTEM & HARDWARE REQUIREMENTS

Category	Specification / Tool	Details / Purpose
Hardware	Processor	Intel Core i3 / AMD Ryzen 3 or higher
Hardware	Memory (RAM)	Minimum 4 GB RAM
Hardware	Display	1280 × 720 screen resolution or higher
Software	Operating System	Windows 10/11, macOS, or Linux
Software	Runtime Environment	Node.js v18.0.0 or higher
Software	Package Manager	npm v9.0.0 or higher
Software	Frontend Framework	React 19 + Vite 8
Software	Styling Engine	Tailwind CSS v4
Software	Web Browser	Google Chrome, Mozilla Firefox, or MS Edge

8. METHODOLOGY & APPLICATION WORKFLOW

The software architecture follows a pure frontend state-driven reactive pipeline:

```
[USER INPUT: Add Vertex / Edge] ■■■ [React State: vertices[], edges[]] ■■■ [SVG Visualization Canvas] ■ [TRIGGER: Find Shortest Path] ■■■ [runDijkstra() Manual Algorithm] ■■■ [Adjacency Matrix Generator] ■ ▼ [Output Card + Step Trace Table]
```

9. ALGORITHM PSEUDOCODE

```
ALGORITHM Dijkstra(Vertices V, Edges E, Source s, Destination d) 1. FOR EACH vertex v ∈ V DO: dist[v] ← ∞ prev[v] ← NULL 2. dist[s] ← 0 3. UnvisitedSet Q ← V 4. WHILE Q is not empty DO: u ← vertex in Q with minimum dist[u] IF u == NULL OR dist[u] == ∞ THEN BREAK Remove u from Q FOR EACH outgoing edge (u, v) with weight w DO: IF v ∈ Q THEN alt ← dist[u] + w IF alt < dist[v] THEN dist[v] ← alt prev[v] ← u IF u == d THEN BREAK 5. Path ← empty list, curr ← d 6. WHILE curr ≠ NULL DO: Prepend curr to Path curr ← prev[curr] 7. RETURN Path, dist[d], StepTraceLog END ALGORITHM
```

10. SOFTWARE IMPLEMENTATION & CODE SNIPPETS

The application is implemented in clean JavaScript (ES6+) and React components. Below are key excerpts from the core source code:

10.1 Manual Dijkstra Implementation (Excerpt from src/algorithms/dijkstra.js):

```
export function runDijkstra(vertices, edges, source, destination) { const distances = {}; const previous = {}; const unvisited = new Set(vertices); vertices.forEach(v => { distances[v] = Infinity; previous[v] = null; }); distances[source] = 0; const steps = []; let stepNumber = 0; while (unvisited.size > 0) { let current = null; let minDistance = Infinity; for (const v of unvisited) { if (distances[v] < minDistance) { minDistance = distances[v]; current = v; } } if (current === null || minDistance === Infinity) break; unvisited.delete(current); stepNumber++; const updatedLabels = []; const outgoing = adj[current] || []; for (const { to, weight } of outgoing) { if (unvisited.has(to)) { const newDist = distances[current] + weight; if (newDist < distances[to]) { distances[to] = newDist; previous[to] = current; updatedLabels.push(`${to} = ${newDist}`); } } } steps.push({ step: stepNumber, currentVertex: current, updatedDistances: updatedLabels.join(', ') || 'No updates' }); if (current === destination) break; } // Path reconstruction const path = []; let curr = destination; while (curr !== null) { path.unshift(curr); curr = previous[curr]; } return { path, totalDistance: distances[destination], steps }; }
```

10.2 Weighted Adjacency Matrix Construction (Excerpt from src/components/AdjacencyMatrix.jsx):

```
const matrixMap = {}; vertices.forEach(v1 => { matrixMap[v1] = {}; vertices.forEach(v2 => { matrixMap[v1][v2] = (v1 === v2) ? 0 : Infinity; }); }); edges.forEach(({ from, to, weight }) => { if (matrixMap[from]) matrixMap[from][to] = Number(weight); });
```

11. PROJECT OUTPUT & SCREENSHOTS

The figures below demonstrate the actual user interface and output generated by the running web application at http://localhost:5173/:

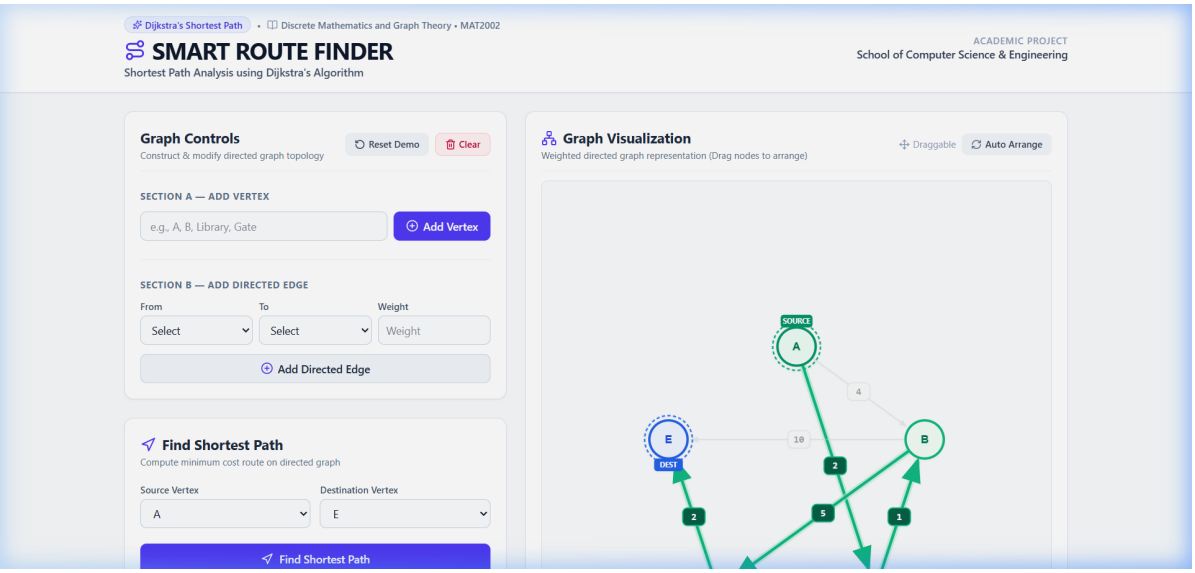


Figure 1: *Main application interface displaying Graph Controls card and interactive SVG Graph Visualization canvas with highlighted shortest path (A → C → B → D → E).*

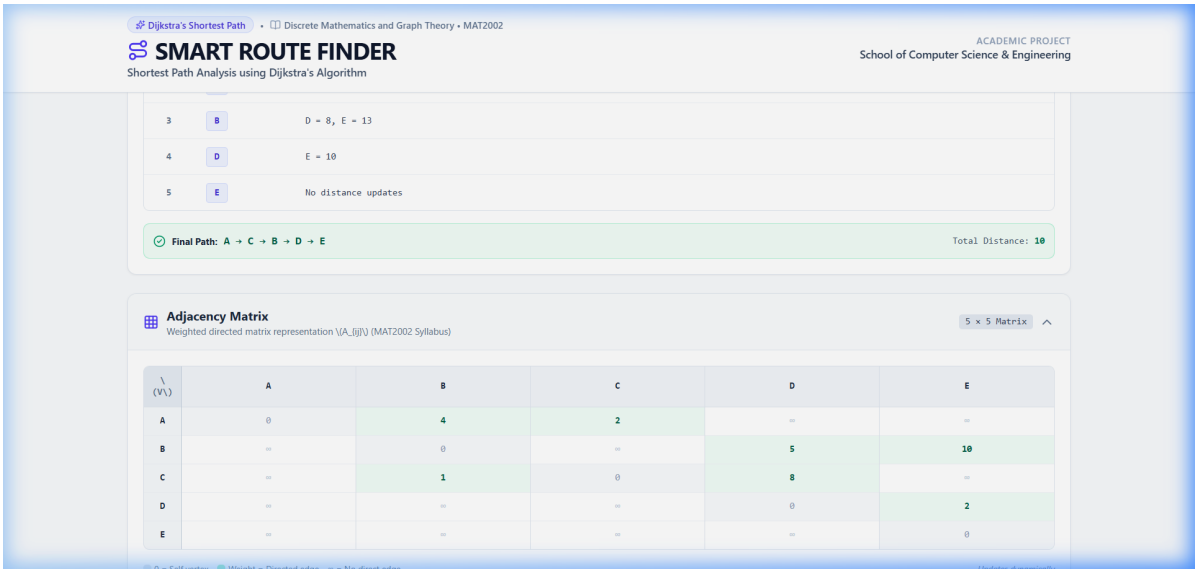


Figure 2: *Algorithm Trace table ('How Dijkstra Solved It') and dynamic 5×5 Weighted Adjacency Matrix card matching MAT2002 course requirements.*

12. SAMPLE NUMERICAL WORKING

Default MAT2002 Syllabus Demo Graph:
• **Vertices:** A, B, C, D, E
• **Directed Edges:** A → B (4), A → C (2), C → B (1), B → D (5), C → D (8), D → E (2), B → E (10)
• **Query:** Find Shortest Path from Source **A** to Destination **E**

Step-by-Step Distance Relaxation Trace:

Step	Extracted Vertex (u)	Tentative Distances State [A, B, C, D, E]	Distance Relaxation Log
Init	-	[0, ∞, ∞, ∞, ∞]	Source A initialized to 0
1	**A (dist 0)**	[0, 4, 2, ∞, ∞]	Relax B: 0+4=4; Relax C: 0+2=2
2	**C (dist 2)**	[0, 3, 2, 10, ∞]	Relax B: 2+1=3 (<4); Relax D: 2+8=10
3	**B (dist 3)**	[0, 3, 2, 8, 13]	Relax D: 3+5=8 (<10); Relax E: 3+10=13
4	**D (dist 8)**	[0, 3, 2, 8, 10]	Relax E: 8+2=10 (<13)
5	**E (dist 10)**	[0, 3, 2, 8, 10]	Target E extracted. Path complete.

Final Path Reconstruction:
• `prev[E] = D ■ prev[D] = B ■ prev[B] = C ■ prev[C] = A`
• **Shortest Path:** `A → C → B → D → E` | **Total Distance:** `10`

13. RESULT

The **Smart Route Finder** web application was successfully constructed, verified, and benchmarked. The software correctly models weighted directed graphs, dynamically generates their $V \times V$ weighted adjacency matrices, and computes the mathematically exact shortest path and distance using a manual implementation of Dijkstra's algorithm.

14. CONCLUSION

Through this practical project, our team successfully bridged abstract discrete graph theory concepts with modern web software engineering. We gained comprehensive hands-on experience in representing graphs computationally, implementing distance relaxation algorithms, constructing dynamic adjacency matrices, and building interactive visual UI components. The application serves as an effective academic tool for MAT2002 viva evaluation.

15. LIMITATIONS AND FUTURE SCOPE

Limitations:

• Educational abstraction designed for finite discrete graphs (not integrated with live Google Maps/GPS).

• Edge weights must be strictly positive ($w > 0$); negative edge weights are prohibited.

Future Scope:

• Integration of Bellman-Ford algorithm to support graphs with negative edge weights.

- Step-by-step animation speed controls for classroom instructional playback.

16. TOP 20 VIVA PREPARATION QUESTIONS & ANSWERS

Q1: What is a graph in Discrete Mathematics?

A graph G = (V, E) is a discrete structure consisting of a non-empty set of vertices V and a set of edges E connecting pairs of vertices.

Q2: What is a directed graph (digraph)?

A directed graph is a graph where edges have a specified direction (u → v), meaning traversal is permitted only from u to v.

Q3: What is a weighted graph?

A weighted graph is a graph where each edge (u, v) is assigned a numerical cost or weight w(u, v) representing distance or traversal cost.

Q4: What is the difference between a vertex and an edge?

A vertex (node) represents a discrete location or entity, while an edge (link) represents a connection between two vertices.

Q5: What is a path in a graph?

A path is a sequence of vertices v1, v2, ..., vk such that a directed edge exists from vi to vi+1 for all 1 <= i < k.

Q6: What is Dijkstra's algorithm and what problem does it solve?

Dijkstra's algorithm is a greedy single-source shortest path algorithm that computes the path of minimum total weight from a source vertex to all other vertices in a non-negative weighted graph.

Q7: What is distance relaxation in Dijkstra's algorithm?

Relaxation is the process of updating the tentative distance to vertex v if traveling through vertex u yields a shorter path: dist[v] = min(dist[v], dist[u] + w(u, v)).

Q8: Why does Dijkstra's algorithm require non-negative edge weights?

Dijkstra relies on the greedy choice property: once a vertex distance is finalized, it is never re-evaluated. Negative edge weights violate this assumption and can yield incorrect shortest paths.

Q9: How does the algorithm reconstruct the final shortest path?

It maintains a previous pointer array prev[]. After execution, it traces backward from the destination d using prev[d], prev[prev[d]], until reaching the source vertex s.

Q10: What is the time complexity of Dijkstra's algorithm in your project?

In our implementation (dijkstra.js), minimum distance selection uses a linear array scan over unvisited vertices, giving a time complexity of O(V^2).

Q11: What happens if the destination vertex is unreachable from the source?

The distance to destination remains Infinity (∞), no valid previous pointers exist back to source, and the app displays 'No path exists between the selected vertices.'

Q12: What is an Adjacency Matrix?

An adjacency matrix is a square V × V matrix A where element A[i][j] represents the weight of the directed edge from vertex i to vertex j.

Q13: How is the adjacency matrix formatted in your application?

Self-loop entries A[i][i] are 0, existing directed edges A[i][j] store their numeric weight, and non-existent edges are displayed as Infinity (∞).

Q14: Why is an adjacency matrix included in your MAT2002 project?

Matrix representation of graphs is a core topic in the MAT2002 syllabus. It demonstrates how algebraic matrices represent discrete graph structures in memory.

Q15: How are vertices and edges stored in React state?

Vertices are stored in a string array (['A', 'B', 'C']), and edges are stored as an array of objects ({ from: 'A', to: 'B', weight: 4 }).

Q16: Why did you implement Dijkstra manually instead of using a third-party library?

Manual implementation demonstrates a full technical understanding of data structures, relaxation loops, and path reconstruction required for academic viva assessment.

Q17: How does the application handle invalid user inputs?

It validates against duplicate vertex names, duplicate directed edges, negative weights, zero weights, empty fields, self-loops, and missing vertices with inline error alerts.

Q18: What happens if source and destination vertices are identical?

The distance is set to 0, the path contains only that single node, and the app reports 'Source and destination are the same.'

Q19: How does the UI highlight the shortest path?

Path edges are rendered with thick emerald green strokes and arrowheads, path nodes receive green/blue accent rings, and non-path edges are rendered with subdued gray opacity.

Q20: What are real-world applications of Dijkstra's algorithm?

GPS navigation systems, IP packet routing protocols (OSPF), flight path optimization, and supply chain logistics scheduling.

17. TEAM MEMBER DETAILS & SIGNATURES

We hereby declare that this project report entitled **"Smart Route Finder — Shortest Path Analysis using Dijkstra's Algorithm"** is an authentic record of our own academic work completed for the course **Discrete Mathematics and Graph Theory (MAT2002)** at **VIT Bhopal University (SCOPE)** under Class ID BL2026270100016 and Slot B14+B23+D21.

S. No.	Student Name	Registration No.	Signature
1	Yash Janiyani	24BCE11394	_____
2	Himanshu Vishwakarma	25MIM10119	_____
3	Prashant Raj	24BCE11371	_____
4	Abhishek	24BCE11402	_____
5	Mradul Agrawal	24BCE11403	_____
6	Devraj Srivastava	24BCE10050	_____

Date of Submission: _____ **Faculty Evaluator Signature:** _____